STATISTICAL COMPUTING JOURNAL

Dr. Homi Bhabha State University - The Institute of Science
SYBSc Business AI, Semester III (2026-27)

All Python codes (Practicals 1 to 10) with outputs

Practical 1

Aim

To create and display a student-score data frame using Python and pandas.

Code

```
import pandas as pd

data = {
    "Student": ["Aarav", "Priya", "Rohit", "Sneha", "Kunal", "Divya"],
    "Score": [78, 85, 92, 67, 74, 88]
}
df = pd.DataFrame(data)
print(df)
```

Output

```
Student  Score
0   Aarav     78
1   Priya     85
2   Rohit     92
3   Sneha     67
4   Kunal     74
5   Divya     88
```

Practical 2

Aim

To calculate the measures of central tendency (mean, median and mode) for the given sales data.

Code

```
import pandas as pd

sales = pd.Series([42, 38, 51, 29, 46, 34, 61, 40, 37, 44])
print("Mean:", sales.mean())
print("Median:", sales.median())
print("Mode:", sales.mode().tolist())
```

Output

```
Mean: 42.2
Median: 41.0
Mode: [29, 34, 37, 38, 40, 42, 44, 46, 51, 61]
```

Practical 3

Aim

To calculate the sample variance and sample standard deviation for the given sales data.

Code

```
import pandas as pd

sales = pd.Series([42, 38, 51, 29, 46, 34, 61, 40, 37, 44])
variance = sales.var()
std_dev = sales.std()
print(f"Sample Variance: {variance:.2f}")
print(f"Sample Std Dev: {std_dev:.2f}")
```

Output

```
Sample Variance: 82.18
Sample Std Dev: 9.07
```

Practical 4

Aim

To generate the Binomial PMF for n = 5 trials and probability of success p = 0.3, and visualize it using a bar chart.

Code

```
import numpy as np
import scipy.stats as stats
import matplotlib.pyplot as plt

n, p = 5, 0.3
k = np.arange(0, n + 1)
pmf = stats.binom.pmf(k, n, p)

for val, prob in zip(k, pmf):
    print(f"P(X={val}) = {prob:.5f}")

plt.bar(k, pmf, color="steelblue", edgecolor="black")
plt.title("Binomial PMF (n=5, p=0.3)")
plt.xlabel("Number of Successes (k)")
plt.ylabel("Probability")
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

Output

```
P(X=0) = 0.16807
P(X=1) = 0.36015
P(X=2) = 0.30870
P(X=3) = 0.13230
P(X=4) = 0.02835
P(X=5) = 0.00243
```

(Run the code to see the plot.)

Practical 5

Aim

To generate the Poisson PMF for an average rate of lambda = 4 complaints per day and plot the probability distribution.

Code

```
import numpy as np
import scipy.stats as stats
import matplotlib.pyplot as plt

lam = 4
k = np.arange(0, 15)
pmf_poisson = stats.poisson.pmf(k, lam)

print(f"P(X=3) = {stats.poisson.pmf(3, lam):.5f}")

plt.bar(k, pmf_poisson, color="orange", edgecolor="black")
plt.title(r"Poisson PMF ($\lambda=4$ complaints/day)")
plt.xlabel("Number of Events (k)")
plt.ylabel("Probability")
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

Output

```
P(X=3) = 0.19537
```

(Run the code to see the plot.)

Practical 6

Aim

To demonstrate how the distribution of sample means becomes more concentrated and approximately normal as sample size increases, using an exponential (non-normal) population.

Code

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
# Generating non-normal skewed population (Exponential)
population = np.random.exponential(scale=500, size=100000)

sample_sizes = [10, 30, 100]
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for i, n in enumerate(sample_sizes):
    sample_means = [np.random.choice(population, size=n, replace=False).mean()
                    for _ in range(1000)]
    print(f"Sample Size n = {n}: Mean={np.mean(sample_means):.1f}, SD={np.std(sample_means):.1f}")
    axes[i].hist(sample_means, bins=30, color="teal", edgecolor="black", alpha=0.7)
    axes[i].set_title(f"Sample Size n = {n}\nMean={np.mean(sample_means):.1f}, SD={np.std(sample_means):.1
    axes[i].set_xlabel("Sample Mean")
    axes[i].set_ylabel("Frequency")

plt.tight_layout()
plt.show()
```

Output

```
Sample Size n = 10: Mean=501.2, SD=159.9
Sample Size n = 30: Mean=495.1, SD=92.8
Sample Size n = 100: Mean=500.1, SD=49.6
```

(Run the code to see the plot.)

Practical 7

Aim

To compute 90%, 95% and 99% confidence intervals for the mean weight using the t-distribution.

Code

```
import numpy as np
from scipy import stats

# NOTE: Replace this with the weight data given in your exam/journal.
# (Placeholder data: 10 weights with mean 250.)
weights = np.array([246, 247, 248, 249, 249, 251, 251, 252, 253, 254])

mean = weights.mean()
se = stats.sem(weights)  # standard error of the mean

for conf in [0.90, 0.95, 0.99]:
    lo, hi = stats.t.interval(conf, df=len(weights) - 1, loc=mean, scale=se)
    print(f"{int(conf * 100)}% CI: ({lo:.2f}, {hi:.2f})")
```

Output

```
90% CI: (248.48, 251.52)
95% CI: (248.12, 251.88)
99% CI: (247.30, 252.70)
```

Note: replace the placeholder data with the weight data from your exam.

Practical 8

Aim

To test whether the mean bottle volume differs significantly from the hypothesized population mean of 500 ml.

Code

```
import numpy as np
from scipy import stats

bottles = np.array([498, 502, 497, 501, 499, 503, 496, 500])
t_stat, p_val = stats.ttest_1samp(bottles, popmean=500)
print(f"t-statistic: {t_stat:.3f}")
print(f"p-value: {p_val:.4f}")
```

Output

```
t-statistic: -0.577
p-value: 0.5818
```

Practical 9

Aim

To determine whether the mean performance of Shift A differs significantly from Shift B.

Code

```
import numpy as np
from scipy import stats

shift_a = np.array([45, 48, 42, 50])
shift_b = np.array([52, 55, 50, 58])
t_stat, p_val = stats.ttest_ind(shift_a, shift_b)
print(f"t-statistic: {t_stat:.3f}")
print(f"p-value: {p_val:.4f}")
```

Output

```
t-statistic: -3.030
p-value: 0.0231
```

Practical 10

Aim

To compute the regression intercept, slope and R-squared for Sales predicted by TV Spend, using ordinary least squares.

Code

```
import numpy as np
import pandas as pd
import statsmodels.api as sm

df = pd.DataFrame({
    "TV_Spend": [100, 150, 200, 250, 300],
    "Sales": [150, 200, 230, 280, 320]
})

# Add constant (intercept) column for OLS
X = sm.add_constant(df["TV_Spend"])
y = df["Sales"]
model = sm.OLS(y, X).fit()

print(f"Intercept (beta_0): {model.params['const']:.2f}")
print(f"Slope (beta_1): {model.params['TV_Spend']:.2f}")
print(f"R-squared: {model.rsquared:.4f}")
```

Output

```
Intercept (beta_0): 68.00
Slope (beta_1): 0.84
R-squared: 0.9955
```